

MUSTAFACODE

Lecture 14

Chapter: Composition in C++ (Detailed Notes)
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

Lecture 14: Chapter: Composition in C++ (Detailed Notes)

Chapter: Composition in C++ (Detailed Notes)

◆ 1. Composition Kya Hoti Hai?

Composition Object Oriented Programming ka ek important concept hai.

Definition

Jab ek class ke andar doosri class ka object banaya jaye to usay Composition kehte hain.

Ye: 🖱️ **Has-A Relationship** ko represent karta hai.

◆ 2. Has-A Relationship

Composition ko hum "Has-A" relationship bhi kehte hain.

Real Life Examples

Object	Has-A
Car	Engine
House	Rooms
Person	Hands
Student	Name
Computer	Keyboard

◆ 3. Simple Example of Composition

```
class Engine{  
};  
  
class Car{  
    Engine e;  
};
```

Explanation

Yahan:

- `Car` class ke andar `Engine` object hai
- Iska matlab:

👉 Car **has a** Engine

Ye Composition hai.

◆ 4. Composition ka Purpose

Composition ka use:

- Code Reuse

- Clean Code
- Better Organization
- Easy Maintenance
- Real World Modeling

ke liye hota hai.

◆ 5. Student Class Without Composition

Pehle hum directly `char *` use kar rahe the.

```
class Student{  
private:  
    char * name;  
};
```

Problem

Hume manually:

- memory allocate
- memory delete
- copy constructor
- deep copy

sab kuch manage karna padta tha.

Code complicated ho jata tha.

◆ 6. Student Class With Composition

Ab hum String class banayenge aur uska object Student class ke andar rakhenge.

```
class Student{  
private:  
    String name;  
};
```

Explanation

Yahan:

👉 Student **has a** String

Ye Composition hai.

◆ 7. String Class

```
class String{  
  
private:  
    char * ptr;  
  
public:  
    String();  
  
    String(const String &);  
  
    void SetString(const char *);  
  
    const char * GetString() const;  
  
    ~String();  
};
```

◆ 8. String Class Explanation

Function	Purpose
Constructor	Object initialize karta hai
Copy Constructor	Deep copy karta hai
SetString	String set karta hai
GetString	String return karta hai
Destructor	Memory free karta hai

◆ 9. Default Constructor

```
String::String(){  
  
    cout << "Constructor::String..\n";  
  
    ptr = NULL;  
}
```

Explanation

Object create hone par:

```
ptr = NULL;
```

kiya jata hai taake garbage value na aaye.

◆ 10. Copy Constructor

```
String::String(const String & str){  
  
    if(str.ptr != NULL){  
  
        ptr = new char[strlen(str.ptr)+1];  
  
        strcpy(ptr, str.ptr);  
    }  
    else{  
        ptr = NULL;  
    }  
}
```

◆ 11. Deep Copy

Deep Copy Kya Hoti Hai?

Jab:

- new memory allocate ho
- aur value copy ki jaye

usay Deep Copy kehte hain.

◆ 12. Deep Copy Example

```
ptr = new char[strlen(str.ptr)+1];  
  
strcpy(ptr, str.ptr);
```

Explanation

- New memory create hoti hai
- Data copy hota hai
- Dono objects ki separate memory hoti hai

◆ 13. Setter Function

```
void String::SetString(const char * str){  
  
    if(ptr != NULL){  
  
        delete [] ptr;  
  
        ptr = NULL;  
    }  
  
    if(str != NULL){  
  
        ptr = new char[strlen(str)+1];  
  
        strcpy(ptr, str);  
    }  
}
```

◆ 14. Setter Function Explanation

Step 1

```
delete [] ptr;
```

Purani memory delete karta hai.

Step 2

```
ptr = NULL;
```

Pointer ko safe banata hai.

Step 3

```
ptr = new char[strlen(str)+1];
```

New memory allocate karta hai.

Step 4

```
strcpy(ptr, str);
```

String copy karta hai.

◆ 15. Memory Leakage

Definition

Jab:

- memory allocate ho jaye
- magar free na ho

to usay Memory Leakage kehte hain.

◆ 16. Memory Leak Example

```
ptr = new char[20];  
  
ptr = new char[50];
```

Pehli memory inaccessible ho gayi.

Ye Memory Leak hai.

◆ 17. Getter Function

```
const char * String::GetString() const{  
  
    return ptr;  
}
```

Explanation

Ye function string return karta hai.

◆ 18. Destructor

```
String::~String(){  
  
    delete [] ptr;  
  
    cout << "Destructor::String..\n";  
}
```

Explanation

Destructor:

- dynamic memory free karta hai
- memory leak se bachata hai

◆ 19. Student Class Using Composition

```
class Student{  
  
private:  
    float gpa;  
  
    int rollNumber;  
  
    String name;  
  
public:  
  
    Student(char * = NULL, int = 0, float = 0.0);  
  
    Student(const Student &);  
  
    void SetName(const char *);  
  
    const char * GetNamePtr() const;  
  
    ~Student();  
};
```

◆ 20. Student Constructor

```
Student::Student(char * _name, int roll, float g){  
  
    cout << "Constructor::Student..\n";  
  
    name.SetString(_name);  
  
    rollNumber = roll;  
  
    gpa = g;  
}
```

◆ 21. Explanation

```
name.SetString(_name);
```

Yahan:

- composed object `name`
- ka function call ho raha hai

◆ 22. Copy Constructor

```
Student::Student(const Student & s){  
  
    name.SetString(s.name.GetString());  
  
    gpa = s.gpa;  
  
    rollNumber = s.rollNumber;  
}
```

◆ 23. Complex Statement Explanation

```
name.SetString(s.name.GetString());
```

Step-by-Step

s.name

Copied object ka composed object access ho raha hai.

GetString()

String value return kar raha hai.

SetString()

New object ke composed object mein value set kar raha hai.

◆ 24. Getter Function

```
const char * Student::GetNamePtr() const{  
  
    return name.GetString();  
}
```

◆ 25. Setter Function

```
void Student::SetName(const char * n){  
  
    name.SetString(n);  
}
```

◆ 26. Student Destructor

```
Student::~~Student(){  
  
    cout << "Destructor::Student..\n";  
}
```

◆ 27. Main Function Example

```
int main(){  
  
    Student *aStudent = new Student("Fakhir", 899, 3.1);  
  
    cout << "Name: "  
        << aStudent->GetNamePtr()  
        << endl;  
  
    delete aStudent;  
}
```

◆ 28. Output

```
Constructor::String..  
Constructor::Student..  
  
Name: Fakhir  
  
Destructor::Student..  
Destructor::String..
```

◆ 29. Constructor Calling Order

Rule

👉 Constructors: From composed object → composing object

Example

```
class Engine{  
};  
  
class Car{  
    Engine e;  
};
```

Order

1. Engine Constructor
2. Car Constructor

◆ 30. Destructor Calling Order

Rule

👉 Destructors: From composing object → composed object

Example

1. Car Destructor
2. Engine Destructor

◆ 31. Why This Happens?

Constructor

Pehle andar wala object banana zaruri hota hai.

Is liye:

- pehle composed object
 - phir composing object
-

Destructor

Pehle outer object destroy hota hai.

Phir andar wala object destroy hota hai.

◆ 32. Accessing Composed Object Methods

Syntax

```
ObjectName.MemberFunction()
```

Example

```
name.SetString("Ali");
```

◆ 33. Important Note About Private Members

Class ke member functions: 👉 private members access kar sakte hain.

Example:

```
gpa = s.gpa;
```

◆ 34. Real Life Example

Example 1: Car & Engine

```
class Engine{  
};  
  
class Car{  
  
private:  
    Engine e;  
};
```

👉 Car has a Engine

Example 2: House & Room

```
class Room{  
};  
  
class House{  
  
private:  
    Room r1;  
    Room r2;  
};
```

👉 House has rooms

Example 3: Computer & Keyboard

```
class Keyboard{  
};  
  
class Computer{  
  
private:  
    Keyboard k;  
};
```

👉 Computer has a Keyboard

◆ 35. Advantages of Composition

Advantage	Explanation
Code Reuse	Existing classes reuse hoti hain
Clean Code	Code readable hota hai
Easy Maintenance	Bugs fix karna easy
Better Design	Real world modeling
Security	Encapsulation improve hoti hai

◆ 36. Important Interview/Paper Questions

Q1: Composition kya hoti hai?

Jab ek class ke andar doosri class ka object ho to usay Composition kehte hain.

Q2: Composition kis relationship ko represent karti hai?

Has-A Relationship

Q3: Constructor kis order mein call hote hain?

*Pehle composed object ke constructors
phir composing object ke constructors*

Q4: Destructor kis order mein call hote hain?

*Pehle composing object ke destructors
phir composed object ke destructors*

◆ 37. Final Summary

Concept	Meaning
Composition	Ek class ke andar doosri class ka object
Has-A	Object doosri cheez rakhta hai
Deep Copy	Separate memory create karna
Memory Leak	Memory free na karna
Constructor Order	Inner → Outer
Destructor Order	Outer → Inner
SetString	String set karta hai
GetString	String return karta hai



End of Chapter

Composition in C++
